

КАСКАД И СМАРТ РОУТИНГ

Система распределения трафика — архитектура и логика

Блок — Каскад · Версия 3.1 · Май 2026

Документ описывает упрощённую логику распределения Pay-In заявок: распрацентовка работает только на уровне методов оплаты, между трейдерами она убрана. Внутри метода реквизиты конкурируют в общей гонке рейтинга. v3.1 добавляет провайдера с собственным %, два варианта розыгрыша по методам, fallback с провайдера и правило сброса рейтинга при изменении лимитов реквизита.

Документ ТЗ — Каскад и смарт роутинг (v3.1)

Назначение	Требования к системе распределения трафика между трейдерами, реквизитами и провайдером
Аудитория	Backend-разработчик, Tech Lead
Статус	Версия 3.1 — добавлен % на провайдера, два варианта розыгрыша по методам (детерминированный и стохастический), fallback с провайдера, сброс рейтинга реквизита при изменении лимитов
Заменяет	Версию 3.0 (Май 2026). Развивает архитектуру v3.0, не ломая её принципы.

1. Общая концепция

Каскад — это система которая автоматически решает на какого трейдера и на какой реквизит назначить входящую Pay-In заявку. Решение принимается в момент создания заявки, за миллисекунды, без участия человека.

Система устроена как упорядоченная цепочка уровней по методам оплаты.

Распределение % трафика задаётся только между методами (Fork / Card / Provider). Внутри каждого метода трейдеры не имеют целевых процентов — реквизиты конкурируют в общей гонке рейтинга.

1.1 Эволюция архитектуры

Эта версия (v3.0) отказывается от концепции распределения между трейдерами, которая была в v1.0 и v2.0. Причина — фундаментальная проблема: при разной мощности трейдеров (количество реквизитов, доступные суммы) распределение создавала виртуальные долги, конфликты и непредсказуемое поведение. Новая архитектура проще, чище и не требует системы скользящих окон между трейдерами.

Вместо распределения введён `trader_multiplier` — индивидуальный коэффициент скорости накопления рейтинга реквизитов трейдера. Это даёт оверу тонкий контроль над приоритетами трейдеров, не вводя сложности виртуальных долгов и пересчётов целевых процентов.

1.2 Уровни каскада

Уровень	Метод	% трафика	Логика выбора реквизита
Уровень 1	FORK	Основной (например 80%)	Полная гонка рейтинга: $idle \times \max(fill_mult, trader_mult) + \text{буст новичка}$
Уровень 2	CARD	Зарезервированный (например 20%)	Упрощённая гонка: $idle \times trader_mult$. Без <code>fill_mult</code> и буста новичка.
Уровень 3	Провайдер	Опционально с %, или только fallback	Внешний API. Может получать резерв % или работать только как финальный fallback.

1.3 Методы оплаты

Метод — характеристика трейдера и его реквизитов. Определяет на каком уровне каскада участвует трейдер.

Метод	Описание	Время обработки	Особенности
CARD	P2P перевод по номеру карты	1–2 минуты	Быстрая верификация. Простая гонка реквизитов.
FORK	P2P через биржевые ордера	7–10 минут	Верификация через чат. Полная рейтинговая гонка с бустом новичка.

2. Распределение трафика между методами

Единственный явный % который задаёт овнер — это распределение трафика между методами/уровнями каскада. Никакой распределения внутри метода между трейдерами не существует.

2.1 Конфигурация

Параметр	По умолчанию	Описание
fork_percent	100	% трафика, идущий первично на Fork.
card_percent	0	% трафика, идущий первично на Card.
provider_percent	0	Новое в v3.1. % трафика, идущий первично на провайдера. Сумма = 100.

Сумма `fork_percent` + `card_percent` + `provider_percent` всегда = 100%. Изменение одного параметра автоматически пересчитывает остальные. UI запрещает сохранять конфигурации, где сумма превышает 100%.

По умолчанию (`Fork=100`, `Card=0`, `Provider=0`) система работает как простой каскад: все заявки сначала на Fork, при отсутствии реквизита — fallback на Card, потом на провайдер.

Резервирование % используется когда овнер хочет гарантировать поток на определённый метод или провайдера.

2.2 Алгоритм розыгрыша — два варианта

Алгоритм определяет на какой уровень каскада попадает каждая входящая заявка первично. Рассматриваются два варианта реализации:

Вариант А — Детерминированный счётчик долга (рекомендуемый)

Система ведёт счётчик «накопленного долга» для каждого уровня. После каждой заявки счётчики обновляются: для уровня который её получил долг уменьшается, для остальных — растёт пропорционально их целевому %. Заявка всегда идёт на уровень с наибольшим положительным долгом.

```
// При каждой заявке:
for each level in [FORK, CARD, PROVIDER]:
    level.debt += level.target_percent / 100
winner = level with max(debt)
winner.debt -= 1 → направить заявку на winner
```

Пример при конфигурации Fork=80%, Card=20%, Provider=0%:

Заявка #	Долг Fork (до)	Долг Card (до)	Победитель	После
1	+0.8	+0.2	Fork	Fork=-0.2, Card=+0.2
2	+0.6	+0.4	Fork	Fork=-0.4, Card=+0.4
3	+0.4	+0.6	Card	Fork=+0.4, Card=-0.4
4	+1.2	-0.2	Fork	Fork=+0.2, Card=0
5	+1.0	+0.2	Fork	Fork=0, Card=+0.2

На 5 заявок: 4 Fork и 1 Card — точно соответствует 80%/20%. Последовательность повторяется циклически. Card никогда не получает «полос простоя», даже на короткой дистанции.

Плюсы Варианта А:

- Точное соблюдение пропорции даже на коротких интервалах (важно если 20% Card должны реально означать каждую 5-ю заявку)
- Card-трейдер с резервом 20% получит свои заявки равномерно во времени
- Простая реализация — несколько счётчиков в Redis

Минусы Варианта А:

- Предсказуемость последовательности (но это не критично для внутренней системы)

Вариант В — Стохастический розыгрыш

При каждой заявке генерируется случайное число от 0 до 100. Сравнивается с накопительными порогами:

```

random_value = случайное число от 0 до 100
если random_value < fork_percent:
    уровень = FORK
иначе если random_value < (fork_percent + card_percent):    уровень =
CARD
иначе:    уровень = PROVIDER

```

На длинной дистанции пропорция стремится к заданной конфигурации, но на коротких интервалах возможны отклонения (например, на 20 заявках реально может быть 18 Fork / 2 Card или 15 Fork / 5 Card).

Плюсы Варианта В:

- Простая логика без хранения состояния
- Естественная для понимания

Минусы Варианта В:

- Возможны «полосы простоя» Card на коротких интервалах
- Card-трейдер с резервом 20% может несколько минут не получать заявки — это плохо для UX трейдера

Выбор и обсуждение с программистом

Оба варианта валидны и приемлемы для системы. Рекомендуется Вариант А (детерминированный) для более точного и предсказуемого соблюдения целевых процентов. Финальное решение принимается совместно с программистом — возможно, есть нюансы реализации (производительность, состояние в распределённой среде, конкуренция), которые повлияют на выбор. Архитектура каскада позволяет переключиться между вариантами без переработки остального функционала.

2.3 Учёт заявок в распраентовке методов

Заявка засчитывается в % того метода, на который она была первично направлена розыгрышем. Если заявка пошла на Fork и осталась на Fork — она учитывается как Fork. Если заявка пошла на Card и попала в fallback на Fork — она всё равно учитывается как Card-заявка (по первичному назначению, не по фактическому приземлению).

Это важно для того чтобы fallback-заявки не исказили соблюдение целевых %. Метод Card не должен «перевыполнить» свой 20% план за счёт fallback с Fork.

3. Уровень Fork — гонка реквизитов

На уровне Fork все активные реквизиты всех активных Fork-трейдеров участвуют в одной общей гонке. Никакого разделения по трейдерам, никаких целевых процентов трейдеров.

3.1 Базовая формула рейтинга

```
idle_time = now - last_assigned_at(если заявок не было: last_assigned_at =
created_at)fill_multiplier = функция от заполненности (см. 3.3)trader_multiplier =
индивидуальный коэффициент трейдера (см. 3.5)effective_multiplier =
max(fill_multiplier, trader_multiplier)effective_rating = idle_time ×
effective_multiplier
```

Реквизит с наибольшим effective_rating получает следующую заявку. После назначения last_assigned_at обновляется на текущий момент, рейтинг обнуляется.

3.2 Момент обновления last_assigned_at

last_assigned_at обновляется в момент назначения заявки на реквизит (статус NEW), а не в момент подтверждения трейдером. То есть как только система выбрала реквизит и создала заявку — рейтинг этого реквизита падает в ноль, даже если трейдер ещё не подтвердил.

Реквизит может одновременно иметь несколько заявок в статусе NEW — назначение одной заявки не блокирует реквизит для получения новых.

3.3 fill_multiplier — ускорение по мере заполнения

Множитель растёт по мере заполнения реквизита, чтобы реквизиты близкие к лимиту получали заявки чаще и закрывались чисто.

Заполнение	fill_multiplier	Статус	Поведение
Нет заявок (новичок)	2.0	Новичок	Буст для быстрого получения первой заявки
0 – 59%	1.0	Обычный	Нейтральное накопление рейтинга
60 – 69%	1.5	Буст	Лёгкое ускорение
70 – 79%	2.0	Ускорение	Заметное ускорение
80 – 89%	3.0	Ускорение	Сильное ускорение

90 – 100%	5.0	Ускорение	Максимальное ускорение для чистого закрытия
-----------	-----	-----------	---

3.4 confirmed_fill — заполненность только по подтверждённым

Для расчёта fill_multiplier используется заполненность только по подтверждённым заявкам:

$$\text{confirmed_fill} = \text{confirmed_amount} / \text{limit_amount}$$

Логика: реквизит у которого 60% объёма уже подтверждено — реально ближе к закрытию и заслуживает ускорения. Реквизит у которого 60% висит в NEW — может ещё откатиться и не должен получать ускорение.

3.5 trader_multiplier — индивидуальный множитель трейдера

trader_multiplier — коэффициент скорости накопления рейтинга, индивидуальный для каждого трейдера. Позволяет овнеру дать одним трейдерам более высокий приоритет относительно других, без введения системы целевых процентов.

Параметр	По умолчанию	Описание
trader_multiplier	1.0	Множитель для каждого трейдера. Задаётся овнером индивидуально. Рекомендованный диапазон: 0.5 — 5.0.

3.6 Правило max() — почему именно так

Формула effective_multiplier = max(fill_multiplier, trader_multiplier) сохраняет цельность двух разных целей:

- trader_multiplier работает на «обычной» стадии (заполненность 0-60%), где fill_multiplier = 1.0. Трейдер с множителем 1.5 получает заметное преимущество в стандартной гонке.
- Когда реквизит близок к закрытию (60%+), fill_multiplier перевешивает trader_multiplier. Логика чистого закрытия работает независимо от того, у какого трейдера реквизит.

Альтернативы — перемножение или сумма — были рассмотрены, но отвергнуты. Перемножение делает реквизиты «приоритетных» трейдеров слишком агрессивными при заполнении 80-90%, что нарушает естественный порядок закрытия. Правило max() даёт «приоритет» только когда он не мешает критической логике.

Пример: трейдер А ($\times 1.5$), реквизит на 30%. $\text{fill_mult} = 1.0$, $\text{trader_mult} = 1.5 \rightarrow \text{max} = 1.5$. Этот реквизит копит рейтинг в 1.5 раза быстрее обычного.

Тот же трейдер А, реквизит на 85%. $\text{fill_mult} = 3.0$, $\text{trader_mult} = 1.5 \rightarrow \text{max} = 3.0$. Здесь fill_mult доминирует — реквизит работает в режиме чистого закрытия как обычно.

3.7 Буст новичка

Реквизит, который ещё не получил ни одной заявки (ни в NEW, ни подтверждённой), получает $\text{fill_multiplier} = 2.0$ (буст новичка). Это позволяет новому реквизиту быстро войти в гонку, не ожидая накопления idle_time с момента создания.

Буст снимается автоматически при назначении первой заявки на реквизит.

В формуле $\text{max}()$ буст новичка работает естественно: если у обычного трейдера новый реквизит, $\text{max}(2.0 \text{ буст}, 1.0 \text{ trader}) = 2.0$. Если у приоритетного трейдера ($\times 1.5$) новый реквизит, $\text{max}(2.0 \text{ буст}, 1.5 \text{ trader}) = 2.0$ — буст всё равно перевешивает, что и нужно.

3.8 Сброс рейтинга при изменении лимитов реквизита

Когда трейдер меняет min_amount или max_amount реквизита — рейтинг этого реквизита сбрасывается. Логика основана на том, что смена диапазона суммы фактически делает реквизит «другим» с точки зрения работы каскада: реквизит начинает принимать заявки которые раньше не мог, и наоборот.

Что именно происходит при изменении min_amount или max_amount :

1. last_assigned_at реквизита обновляется на текущий момент $\rightarrow \text{idle_time}$ сбрасывается в 0
2. Реквизит включается в общую гонку с нуля — будет накапливать рейтинг заново
3. Финансовые данные (confirmed_amount , txConfirmed , и т.д.) НЕ сбрасываются — они отражают историю реквизита
4. Если на момент смены лимитов реквизит был новичком ($\text{txPending}=0$ и $\text{txConfirmed}=0$) — он остаётся новичком и сохраняет буст
5. Если на реквизите уже были заявки — буст новичка НЕ возвращается, только idle обнуляется

Решение не возвращать буст новичка при смене лимитов осознанное — это защита от злоупотреблений (трейдер мог бы постоянно менять лимиты, чтобы реквизит каждый раз получал максимальный приоритет). Обнуление idle даёт реквизиту справедливый «второй шанс» в гонке, но не больше.

Что НЕ триггерит сброс рейтинга:

- Изменение `limit_amount` — не меняет какие заявки реквизит может принимать, только когда он закончится
- Изменение `limit_transactions` — аналогично
- Включение / отключение реквизита через `is_active` — выключение просто исключает реквизит из гонки, при включении `idle` продолжает считаться от последнего `last_assigned_at`
- Изменение `accepts_other_banks` или других атрибутов кроме `min_amount/max_amount`

Триггер реализуется в обработчике UPDATE реквизита:

```
при UPDATE реквизита:      если в запросе изменился min_amount ИЛИ max_amount:
requisite.last_assigned_at = now()      обновить Redis-кэш реквизита      записать
в historylog запись о сбросе рейтинга
```

3.9 Сценарий — параллельное заполнение

Треjder добавил 3 реквизита с интервалом ~1 минута: R1 в T+0, R2 в T+1, R3 в T+2. Все имеют буст новичка $\times 2.0$. Все принадлежат трейдеру с `trader_mult = 1.0`.

Заявка в T+5: `idle` у R1 = 5 мин, R2 = 4 мин, R3 = 3 мин. `effective_mult` у всех = $\max(2.0 \text{ буст}, 1.0) = 2.0$. R1 выигрывает по `idle`. Назначаем на R1, буст снимается.

Заявка в T+8: R1 имеет `idle` 3 мин $\times \max(1.0 \text{ fill}, 1.0 \text{ trader}) = 3$. R2 имеет `idle` 7 мин $\times 2.0$ (новичок) = 14. R3 имеет `idle` 6 мин $\times 2.0$ (новичок) = 12. R2 выигрывает.

Заявка в T+11: R1 имеет `idle` 6 мин $\times 1.0 = 6$. R2 имеет `idle` 3 мин $\times 1.0 = 3$. R3 имеет `idle` 9 мин $\times 2.0 = 18$. R3 выигрывает.

Заявка в T+14: R1 имеет `idle` 9 $\times 1.0 = 9$. R2 имеет `idle` 6 $\times 1.0 = 6$. R3 имеет `idle` 3 $\times 1.0 = 3$. R1 выигрывает. Цикл замкнулся.

3.10 Сценарий — приоритетный трейдер

Два трейдера на Fork: A с `trader_mult = 1.5`, B с `trader_mult = 1.0`. У обоих по 2 реквизита с одинаковыми лимитами. Все реквизиты заполняются с нуля параллельно.

На длинной дистанции при стабильной нагрузке трейдер A будет получать заявки примерно в 1.5 раза чаще чем B (пока реквизиты в стадии 0-60%). При заполнении выше 60% действует `fill_multiplier`, и приоритет естественно смещается на реквизит ближайший к лимиту, независимо от владельца.

4. Уровень Card — упрощённая гонка

Card-уровень используется в двух режимах:

- Зарезервированный поток — овнер выделяет конкретный % трафика, который идёт сразу на Card, минуя Fork.
- Fallback с Fork — заявки, для которых Fork не нашёл подходящий реквизит, перенаправляются на Card.

4.1 Логика выбора реквизита на Card

На Card нет fill_multiplier и нет буста новичка. Формула простая:

$$\text{effective_rating} = \text{idle_time} \times \text{trader_multiplier}$$

Среди всех подходящих Card-реквизитов выбирается тот, у кого наибольший effective_rating.

4.2 trader_multiplier на Card

Множитель трейдера задаётся индивидуально каждому Card-трейдеру, аналогично Fork. По умолчанию 1.0. Это даёт овнеру опциональный контроль над приоритетом конкретных Card-трейдеров.

Опциональность: даже если все Card-трейдеры имеют trader_mult = 1.0 (по умолчанию), выбор идёт по чистому idle. Овнер не обязан конфигурировать множители Card-трейдеров — это инструмент при необходимости.

4.3 Почему на Card нет fill_multiplier

Card-уровень — менее критичный по объёму, чем Fork (он получает только зарезервированный % и fallback). Полная рейтинговая система с fill_multiplier излишне сложна для этого случая. Card работает по принципу «бери того кто дольше всех ждал», и этого достаточно.

В будущем при необходимости можно ввести fill_multiplier и для Card — архитектура это позволяет без переработки.

4.4 Учёт заявок

Любая заявка попавшая на Card-трейдера учитывается в его статистике (для аналитики). Однако в логику выбора эта статистика не влияет — нет распрацентовки между Card-трейдерами.

5. Уровень Провайдер

Провайдер — внешний платёжный сервис, подключаемый по API. В v3.1 провайдер может работать в двух режимах: только как финальный fallback (по умолчанию), или с собственным зарезервированным % трафика.

5.1 Режимы работы

Режим	Описание
Только fallback (<code>provider_percent = 0</code>)	По умолчанию. Провайдер получает только те заявки, для которых Fork и Card не нашли реквизит. Не участвует в розыгрыше первичного уровня.
С резервом (<code>provider_percent > 0</code>)	Овнер выделяет провайдеру конкретный % трафика. Эти заявки идут на провайдера напрямую, минуя Fork и Card.

5.2 Когда полезен резерв на провайдера

- Прогрев / разгрузка трейдеров — часть трафика отправляется на провайдера чтобы снизить нагрузку
- Тестовый режим — небольшой % (например 5%) идёт на провайдера для проверки интеграции
- Контракт с провайдером — у провайдера может быть требование минимального объёма для сохранения тарифа
- Резерв на случай инцидентов — провайдер «прогрет» постоянным потоком и готов принять большой объём при необходимости

5.3 Fallback с провайдера

Если заявка попала на провайдера (через резерв или через fallback с Fork/Card) и провайдер не принял заявку (API ошибка, превышен лимит провайдера, отказ) — заявка не сразу проваливается в NO_REQUISITE.

Логика fallback с провайдера:

```

провайдер вернул ошибку или отказ: |— заявка ещё не пробовала Fork → попробовать
Fork|   |— Найдено реквизит → НАЗНАЧИТЬ → СТОП|   |— Не найден → попробовать
Card| |— заявка ещё не пробовала Card → попробовать Card|   |— Найдено реквизит →
НАЗНАЧИТЬ → СТОП|   |— Не найден → NO_REQUISITE|   |— заявка уже пробовала Fork и
Card → NO_REQUISITE

```

Это даёт заявке максимальный шанс найти подходящий путь. Резерв на провайдера — «первичная попытка», а не «жёсткое назначение». Логика симметрична поведению резерва на Card.

5.4 Особенности

- Внутри уровня провайдера нет рейтинговой логики — это просто вызов внешнего API
- Если на уровне Провайдера несколько внешних сервисов — между ними может быть распрацентовка по % (аналогично методам), но это выходит за рамки текущей версии
- Заявка на провайдера учитывается в его % статистике независимо от того, прошла она через резерв или через fallback (для учёта объёма по провайдеру)
- Если заявка попала на провайдера через резерв (как первичная) — она засчитывается в provider_percent план
- Если заявка попала на провайдера через fallback (с Fork или Card) — она НЕ засчитывается в provider_percent план

5.5 Если провайдер не настроен

Если provider_percent > 0, но провайдер не настроен или временно отключён — система должна:

6. Не допускать сохранения конфигурации provider_percent > 0 если провайдер не настроен (UI-валидация)
7. Если провайдер был настроен, но временно недоступен — заявки попадающие в резерв провайдера автоматически проваливаются в fallback на Fork и Card
8. Если все уровни не приняли — статус NO_REQUISITE

6. Полный алгоритм назначения заявки

При входящей Pay-In заявке выполняется единый алгоритм маршрутизации. Все шаги выполняются синхронно за миллисекунды.

6.1 Определение первичного уровня

Используется один из двух алгоритмов розыгрыша (см. раздел 2.2):

- Вариант А — детерминированный счётчик долга (рекомендуемый)
- Вариант В — стохастический розыгрыш по накопительным порогам

Результат розыгрыша — один из трёх уровней: FORK, CARD или PROVIDER. На этот уровень заявка идёт первой.

6.2 Маршрут заявки — полная цепочка fallback

Заявка попадает на первичный уровень и далее проходит по цепочке fallback в случае неудачи. Логика fallback симметрична для всех уровней — каждая заявка имеет максимум 3 попытки (по числу уровней).

```

Заявка попала на FORK (первично) |— Fork: выбрать лучший реквизит среди всех
Fork-реквизитов |   |— Найден → НАЗНАЧИТЬ → СТОП |   |— Не найден → попробовать
Card |   |— Card: выбрать лучший реквизит |   |— Найден → НАЗНАЧИТЬ →
СТОП |   |   |— Не найден → попробовать Провайдера |   |— Принял
→ СТОП |   |   |— Не принял → NO_REQUISITE
Заявка попала на CARD
(первично) |— Card: выбрать лучший реквизит |   |— Найден → НАЗНАЧИТЬ → СТОП |
|— Не найден → попробовать Fork |   |— Fork: выбрать лучший реквизит |
|— Найден → НАЗНАЧИТЬ → СТОП |   |— Не найден → попробовать
Провайдера |   |— Принял → СТОП |   |— Не принял →
NO_REQUISITE
Заявка попала на PROVIDER (первично, через резерв) |— Provider: вызвать
API |   |— Принял → СТОП |   |— Не принял → попробовать Fork |   |— Fork:
выбрать лучший реквизит |   |— Найден → НАЗНАЧИТЬ → СТОП |   |—
Не найден → попробовать Card |   |— Найден → НАЗНАЧИТЬ → СТОП |
|   |— Не найден → NO_REQUISITE

```

Принцип: какой бы уровень ни был первичным — заявка пробует оставшиеся уровни в порядке Fork → Card → Provider если они ещё не были опробованы.

6.3 Что значит «нет подходящего реквизита»

Реквизит не подходит когда выполняется хотя бы одно условие:

- `is_active = false` (реквизит вручную отключён или достиг лимита)
- Трейдер реквизита неактивен
- `confirmed_amount + pending_amount >= limit_amount`
- Количество транзакций достигло `limit_transactions`
- Сумма заявки меньше `min_amount` или больше `max_amount` реквизита

6.4 Выбор лучшего реквизита (Fork)

9. Отфильтровать все Fork-реквизиты по условиям подходимости (см. 6.3).
10. Для каждого подходящего реквизита вычислить `effective_rating = idle × max(fill_mult, trader_mult)`.
11. Отсортировать по убыванию `rating`. Взять первый.

6.5 Выбор лучшего реквизита (Card)

12. Отфильтровать все Card-реквизиты по условиям подходимости.
13. Для каждого вычислить `effective_rating = idle × trader_mult`.
14. Отсортировать по убыванию `rating`. Взять первый.

6.6 Действия при успешном назначении на трейдера

15. Заблокировать реквизит в Redis (distributed lock на время операции).
16. Создать заявку со статусом NEW.
17. Увеличить pendingAmount и txPending у реквизита.
18. Обновить last_assigned_at реквизита на текущий момент.
19. Записать в order_assignment_log (для аналитики).
20. Отправить webhook мерчанту о создании заявки.
21. Уведомить трейдера в кабинете.

6.7 Действия при подтверждении трейдером

22. Заявка переходит в статус SUCCESS/CONFIRMED.
23. confirmed_amount += amount, pending_amount -= amount.
24. txConfirmed += 1, txPending -= 1.
25. Пересчитать confirmed_fill реквизита и его fill_multiplier.
26. Если confirmed_amount >= limit_amount → автоматически отключить реквизит (is_active = false).
27. Отправить webhook мерчанту о подтверждении.

6.8 Действия при назначении на провайдера

28. Вызвать API провайдера с параметрами заявки.
29. При успехе — записать в order_assignment_log запись с trader_id = NULL, provider_id = X.
30. При ошибке — пометить заявку как «не принята провайдером» и запустить fallback на следующий уровень (см. 6.2).

7. Модель данных

7.1 Изменения в таблице traders

Поле	Тип	Статус	Описание
method	ENUM	Без изменений	FORK, CARD (расширяемо)
trader_multiplier	DECIMAL	Новое	Индивидуальный множитель скорости рейтинга. По умолчанию 1.0.
traffic_percent	DECIMAL	Удалено	Концепция целевого % трейдера больше не используется

7.2 Поля реквизита в Redis-кэше

Следующие поля вычисляются динамически и хранятся в Redis. В БД пишется только сам факт назначения и подтверждения через лог заявок.

Redis-поле	Описание
idle_time	Вычисляется: now - last_assigned_at. Базовый компонент рейтинга.
confirmed_fill	confirmed_amount / limit_amount. Используется для расчёта fill_multiplier.
fill_multiplier	Множитель на основе confirmed_fill (1.0 — 5.0) или 2.0 для новичка.
effective_multiplier	max(fill_multiplier, trader_multiplier) для Fork, или trader_multiplier для Card.
effective_rating	idle_time × effective_multiplier. Финальное значение для сортировки.
is_new_requisite	Флаг: реквизит не имеет ни одной назначенной или подтверждённой заявки.

7.2.1 Состояние счётчиков долга уровней (для Варианта А)

Если используется детерминированный алгоритм розыгрыша (Вариант А), в Redis хранятся счётчики долга для каждого уровня каскада:

Redis-ключ	Описание
cascade:debt:fork	Текущий долг уровня Fork (DECIMAL)
cascade:debt:card	Текущий долг уровня Card (DECIMAL)
cascade:debt:provider	Текущий долг уровня Provider (DECIMAL)

При изменении конфигурации (fork_percent / card_percent / provider_percent) счётчики долга сбрасываются в 0 чтобы не накопить искажений от старой пропорции.

7.3 Таблица cascade_settings

Поле	Тип	По умолчанию	Описание
fork_percent	DECIMAL	100	% трафика первично на Fork
card_percent	DECIMAL	0	% трафика первично на Card.

provider_percent	DECIMAL	0	Новое в v3.1. % трафика первично на Провайдера. Сумма всех = 100.
distribution_algorithm	ENUM	DETERMINISTIC	Новое в v3.1. Алгоритм розыгрыша: DETERMINISTIC (A) или STOCHASTIC (B). Подлежит обсуждению с программистом.
new_requisite_boost	DECIMAL	2.0	Множитель для реквизитов-новичков
acceleration_threshold	DECIMAL	0.60	Порог заполненности (0–1) с которого включается ускорение
multipliers_config	JSON	см. ниже	Пороги и множители fill_multiplier
<i>sliding_window_hours (старое)</i>	INTEGER	Удалено	Не нужно — нет распрацентовки трейдеров

Формат multipliers_config:

```
[ { "from": 0.0, "to": 0.6, "multiplier": 1.0 }, { "from": 0.6, "to": 0.7, "multiplier": 1.5 }, { "from": 0.7, "to": 0.8, "multiplier": 2.0 }, { "from": 0.8, "to": 0.9, "multiplier": 3.0 }, { "from": 0.9, "to": 1.0, "multiplier": 5.0 } ]
```

7.4 Лог назначений — order_assignment_log

Лог сохраняется для аналитики и отчётности. В алгоритме не используется.

Поле	Тип	Описание
id	UUID	Первичный ключ
trader_id	UUID NULL	Трейдер которому назначена заявка. NULL если заявка ушла на провайдера.
provider_id	UUID NULL	Новое в v3.1. Провайдер на которого ушла заявка. NULL если назначена на трейдера.
order_id	UUID	ID заявки
method	ENUM NULL	Метод трейдера: FORK / CARD. NULL для провайдера.

primary_bucket	ENUM	На какой уровень заявка была направлена первично: FORK / CARD / PROVIDER. Используется для метрики соблюдения целевых %.
final_level	ENUM	Новое в v3.1. На каком уровне заявка реально приземлилась: FORK / CARD / PROVIDER.
is_fallback	BOOLEAN	True если заявка пришла на этот метод как fallback с другого метода (primary_bucket != final_level)
amount	DECIMAL	Сумма заявки
created_at	TIMESTAMP	Время назначения

8. Управление каскадом

8.1 Уровень овнера / администратора

Параметр	Описание
fork_percent / card_percent / provider_percent	Распределение трафика между методами и провайдером. Сумма = 100%.
distribution_algorithm	Выбор алгоритма розыгрыша: DETERMINISTIC (детерминированный счётчик) или STOCHASTIC (случайный). Решается совместно с программистом на этапе реализации.
trader_multiplier	Индивидуальный множитель для каждого трейдера. Задаётся отдельно для каждого, включая Card-трейдеров.
method трейдера	FORK или CARD
is_active трейдера	Включён ли трейдер в систему. Неактивный трейдер не получает заявки.
new_requisite_boost	Множитель буста новичка
acceleration_threshold + multipliers_config	Пороги и множители ускорения fill_multiplier

8.2 Уровень трейдера

Трейдер управляет только своими реквизитами. Никаких настроек на уровне трейдера он не имеет — даже trader_multiplier настраивает овнер.

Настройка реквизита	Описание
<code>min_amount / max_amount</code>	Диапазон суммы заявок которые может принимать реквизит
<code>limit_amount</code>	Максимальный суммарный объём подтверждённых заявок
<code>limit_transactions</code>	Максимальное количество транзакций
<code>is_active</code>	Вручную включить / выключить реквизит
<code>accepts_other_banks</code>	Принимает ли переводы из других банков

9. Расширяемость

9.1 Добавление нового метода оплаты

Шаги добавления нового метода (например QUASI):

31. Добавить значение QUASI в ENUM поля `method` таблицы `traders`.
32. Решить какая логика выбора реквизита применима для нового метода (Fork-style с `fill_mult` или Card-style без).
33. При создании нового уровня — добавить запись в `cascade_levels` с порядковым номером приоритета.
34. Существующий код алгоритма не меняется — он работает с уровнями обобщённо.

9.2 Добавление провайдера или нескольких провайдеров

Текущая v3.0 описывает один провайдер на финальном уровне. В будущем возможны варианты:

- Несколько провайдеров с распрацентовкой % между ними.
- Провайдеры на разных приоритетах в каскаде.
- Провайдер на уровне выше Card — для случаев когда внешний сервис выгоднее Card-трейдеров.

9.3 Введение `fill_multiplier` на Card

Если в будущем нагрузка на Card-уровень вырастет — можно ввести `fill_multiplier` и для Card-реквизитов. Архитектура это позволяет без переработки. Решение принимается отдельно при необходимости.

9.4 Рейтинговая система трейдеров (перспектива)

Trader_multiplier можно сделать динамическим — вычислять автоматически на основе KPI (конверсия, скорость подтверждения, количество апелляций) вместо ручной настройки овнера. Это позволит создать самоадаптирующуюся систему приоритетов.

10. Аналитика и метрики

Поскольку логика выбора не использует скользящих окон, всё что связано со статистикой — это отдельный аналитический слой для людей.

10.1 Метрики для овнера / администратора

- Соблюдение целевых % методов: фактическое распределение primary_bucket по сравнению с fork_percent / card_percent.
- Распределение заявок по трейдерам: количество и объём за период.
- Количество fallback-заявок (Fork→Card и Card→Fork) — индикатор того что мощности на уровнях не сбалансированы.
- Количество заявок ушедших на провайдера и в NO_REQUISITE — индикатор перегрузки или недостатка реквизитов.
- Среднее время от назначения до подтверждения по каждому трейдеру.

10.2 Метрики для трейдера

- Количество и объём своих заявок за период.
- Среднее время подтверждения.
- Заполнение каждого реквизита (подтверждённое и в NEW).
- Свой trader_multiplier (только для просмотра, изменяет овнер).

Детальная проработка дашбордов аналитики выходит за рамки текущей версии ТЗ.

11. Открытые вопросы и направления

Темы для отдельной проработки:

- Выбор финального алгоритма розыгрыша по уровням (Вариант А vs Вариант В) — обсуждается с программистом на этапе реализации

- Гибкая система маршрутизации трафика — правила «мерчант → трейдер», «направление → метод», sticky-маршрутизация для нетинга, группы трейдеров. Серьёзная функциональность, требует отдельного блока ТЗ.
- Логика умных автолимитов (auto_min / auto_max) — была в v1.0, вынесена для отдельной проработки в новой парадигме.
- Таблица покрытия по суммам и её интеграция с маршрутизацией.
- Возможный ввод fill_multiplier для Card при росте нагрузки.
- Динамический trader_multiplier на основе KPI трейдера.
- Возможность ручного override маршрутизации (отправить заявку на конкретного трейдера в обход алгоритма).
- Поведение системы при одновременных конкурентных запросах на один реквизит — distributed lock и точная семантика.
- Распределение между несколькими провайдерами на финальном уровне.
- Алерты при систематическом недополучении заявок Card-трейдером с резервом.
- Pay-Out маршрутизация и её отличия от Pay-In.

Документ ТЗ — Каскад и смарт роутинг, версия 3.1

Заменяет версии 1.0 (Апрель 2026), 2.0 и 3.0 (Май 2026)

Следующие документы серии: Жизненный цикл заявки, Кабинеты, Аналитика